BICT 3202 — Examiner's Marking Guide

End-of-semester examination papers A and B

Confidential: examiner/lecturer use; do not issue with the candidate papers.

Administrative assumptions to confirm before use

The course outline assigns **60% to the examination** and **40% to assessment**, but does not prescribe an examination duration, paper structure or total raw marks. These two draft papers therefore use a proposed **three-hour, 100-mark** format. Confirm the duration, permitted materials, academic-integrity wording, timetable and any Faculty examination rules before printing. The outline lists a 17-week module duration while the teaching files are organised as Weeks 1–16; confirm the approved teaching calendar, but retain the full stated course scope in the examination.

The candidate papers total **100 raw marks**. Apply the examination's 60% course weighting once, in the normal course-grade calculation; do not multiply the paper mark by 60% and then weight it a second time.

General marking principles

- Accept any technically sound, clearly labelled UML model that satisfies the scenario. Do not require one canonical class design when more than one design is valid.
- Award marks for semantics and consistency, not artistic quality. A neat diagram with incorrect logic should not receive full credit.
- Award partial credit for correct modelling elements even when the complete diagram is unfinished.
- Accept justified assumptions if they do not contradict stated business rules. Where an assumption changes a rule, award no credit for the conflicting part.
- Deduct only once for a repeated naming/notation error when it is clearly the same mistake; do not cascade penalties across every model view.
- A <<include>> or <<extend>> relationship is not compulsory when no clear relationship is justified. Credit a candidate who correctly omits it and explains why.
- For non-functional requirements, credit a measurable target or an operationally testable acceptance criterion. “The system must be fast/secure/easy to use” without a measure is not fully testable.

Course-coverage blueprint

Course coverage	Paper A	Paper B
Business needs, stakeholders, functional and non-functional needs	Q1(a–b)	Q1(a–b)
Object modelling, objects/classes, UML building blocks and notation	Q2	Q2
UML relationships, multiplicity, generalisation and inheritance	Q1(d), Q2	Q1(d), Q2
Events, states, operations, concurrency and behavioural views	Q1(e–f), Q3	Q1(e–f), Q3
Use cases, interactions and analysis documentation	Q1(c–e), Q3(c)	Q1(c–e), Q2(c), Q3(c)
Unified Process phases, iterations, workflows, risks and artefacts; reuse	Q4	Q4
Analysis-to-design refinement, cohesion/coupling, architecture and components	Q5	Q5
Model consistency, traceability and CASE-tool support	Q1(d–e), Q5(c)	Q1(d–e), Q5(c)

These are public practice papers, not secure live examination papers. "Confidential" labels mean the examiner guide must be separated from student practice copies; they do not make this public repository private. A live examination requires new, unseen questions and controlled storage outside this repository.

The Section B choice is intentional. Q1 is compulsory so every candidate demonstrates integrated applied modelling; the other questions offer controlled choice across the syllabus.

Paper A — marking points

Question 1 — Community Clinic Appointment System (40 marks)

(a) Business problem, need and success criteria [4]

- 1: identifies the current problem (paper/telephone scheduling, conflicts, manual follow-up or equivalent).
- 1: states a business need/improvement rather than jumping straight to a technology solution.
- 1 + 1: two measurable success criteria, for example reduced double-bookings, shorter booking time, fewer missed appointments or a defined availability/response target. Accept any realistic measure with a baseline/target or clear test.

(b) Requirements [6]

- 1 each, up to 3: distinct, system-oriented functional requirements grounded in the case (request/cancel appointment, manage availability, record visit status, send reminder, enforce non-overlap, etc.).
- 1.5 each, up to 3: two relevant, measurable non-functional requirements (e.g. response time under a stated load; role-based access and an auditable access rule; availability target; notification delivery objective). Award partial credit for a relevant quality attribute without a measurable criterion.

(c) Use-case diagram [6]

- 2: appropriate actors, including the external SMS Gateway and suitable human roles.
- 2: meaningful use cases covering the key appointment goals.
- 1: clear system boundary and valid actor/use-case associations.

- 1: justified <<include>>/<<extend>> or correct omission of either where no relationship is warranted.

(d) Class diagram [12]

- 4: relevant classes (e.g. Patient, Clinician, Appointment, AvailabilitySlot, and justified service/controller or notification class).
- 4: valid associations and multiplicities that reflect the scenario.
- 2: useful typed attributes and operations/responsibilities.
- 2: explicit non-overlap invariant for non-Cancelled appointments ([start, end) permits back-to-back bookings; cancellation frees the time but retains history) and consistency with the sequence/state views.

(e) Sequence diagram [8]

- 1: suitable actor/object lifelines.
- 2: logically ordered request, validation and response messages.
- 2: availability/conflict checks and atomic reservation/persistence (accept a transaction note or combined fragment).
- 1: appointment is created and persisted only on success.
- 1: alternative rejection path (including the no-longer-available case).
- 1: confirmation notification is sent only after commit and is represented consistently with the design.

(f) Appointment state machine [4]

- 1: correct initial node and sensible lifecycle states.
- 1: meaningful state transitions and event/trigger labels; check-in precedes consultation and completion. Accept a staff markNoShow event guarded by time >= start and no arrival (late check-in is permitted only while still Confirmed), or an explicitly justified timed event consistent with authorised staff recording status.
- 1: patient cancellation from Confirmed correctly applies startTime − systemReceiptTime >= 24 elapsed hours, including equality; later referral does not automatically cancel (receptionist can cancel Confirmed only before startTime).
- 1: completion/no-show/cancellation outcomes are distinct and consistent with the case. Atomic state changes reject stale events if check-in and No-show compete. A late-cancellation referral is a review request, not another appointment.

Question 2 — Object modelling and UML foundations (20 marks)

(a) Class/object and model/diagram [4] — 2 marks per pair: 1 for the accurate distinction and 1 for a relevant example.

(b) UML building blocks and well-formedness [4]

- 3: explains things, relationships and diagrams (1 each).
- 1: explains that a well-formedness rule constrains a model so its elements/relationships are meaningful or consistent, and gives one valid example/check.

(c) Association, aggregation and composition [6] — 2 marks per relationship: correct meaning and a justified example; for aggregation/composition, credit reasoning about independent existence, ownership or lifecycle.

(d) UML metamodel levels and extensibility [4]

- 2: M0 runtime/real instances, M1 user model, M2 UML metamodel, M3 meta-metamodel (0.5 each).
- 1: correctly places the running appointment object at M0, the appointment-system model at M1 and the UML metamodel at M2.
- 1: names a valid extension mechanism (stereotype, tagged value or constraint) and states its purpose.

(e) Abstraction and encapsulation [2] — 1 mark per concept for an accurate definition and suitable short example.

Question 3 — Dynamic and behavioural modelling (20 marks)

(a) State, event and operation [4] — 1 mark each for correct meaning; 1 mark for coherent examples/distinction.

(b) Cancellation activity diagram [8]

- 1: initial/final semantics and activity scope are clear.
- 2: ordered actions identify the request and the confirmed appointment.
- 2: decision/guard correctly separates requests made at least 24 hours before the start from later requests.
- 1: eligible cancellation updates the appointment while retaining its history.
- 1: late request is referred to the receptionist rather than automatically cancelled.

- 1: control flow and alternative paths are readable and consistent.

(c) Diagram comparisons [4]

- 1: sequence diagram's ordered messages/lifelines.
- 1: communication diagram's object links and numbered messages.
- 1: activity diagram's workflow/actions and control flow.
- 1: a suitable use/context for each distinction (credit coherent comparative prose).

(d) Concurrency [4]

- 1: recognises competing requests for one remaining time slot.
- 1: uses an appropriate guard/condition.
- 1: proposes an atomic reservation/commit or equivalent concurrency-safe mechanism.
- 1: states the invariant: at most one active/confirmed appointment occupies a clinician's time slot (or equivalent non-overlap rule).

Question 4 — Unified Software Development Process (20 marks)

(a) Four phases [8] — 2 marks per phase: accurate purpose plus a suitable decision/outcome (scope/business case; risk-resolved architectural foundation; implementation/integration/testing; deployment, acceptance and operational transition).

(b) Iterative project plan [8]

- 3: coherent iterations/increments with feedback and a plausible release sequence.
- 2: two relevant workflows, such as requirements, analysis/design, implementation, test, deployment, configuration/change or project management.
- 1: identifies and prioritises a meaningful early risk (SMS integration, privacy/security, availability or scheduling concurrency).
- 2: two relevant artefacts/deliverables and how they help evaluation (e.g. use-case model, risk list, architecture prototype, executable increment, test results, deployment plan).

(c) Reuse [4]

- 3: two suitability checks (1.5 each), with a valid check and a brief explanation of its relevance (e.g. security/privacy, compatibility, quality, licensing, support/maintainability or functional fit).
- 1: a plausible benefit (reduced duplication/effort, faster delivery or consistency).

Question 5 — From analysis to design and architecture (20 marks)

(a) Analysis, design and traceability [4]

- 2: distinguishes understanding the problem/required behaviour from structuring an implementable solution.
- 2: traces the requirement through a use case to a plausible design class and operation.

(b) Responsibility and design quality [8]

- 1: identifies low cohesion/too many unrelated responsibilities in `ClinicSystemManager`.
- 1: identifies high coupling/direct UI-to-everything/database dependencies.
- 3: proposes a sensible responsibility split (e.g. boundary/controller, appointment service, appointment/domain model, repository, notification adapter) with correct ownership.
- 1: explains how the refactoring improves cohesion, coupling, testability or changeability.
- 2: discusses one relevant strength and one limitation/risk of the object-oriented approach in this case (1 each).

(c) Architecture, components and CASE [8]

- 2: suitable layers and responsibilities (presentation, application/service, domain, persistence or an equivalent design).
- 2: meaningful component boundaries and valid dependency direction.
- 1: interfaces/ports isolate change-prone external services such as SMS or persistence.
- 1: explains adaptability/reuse without overclaiming.
- 2: two relevant CASE-tool capabilities (shared model/repository, diagram synchronisation, traceability, validation, documentation or change-impact support).

Paper B — marking points

Question 1 — Campus Equipment Lending System (40 marks)

(a) Business problem, need and success criteria [4] — use the same 1 + 1 + 1 + 1 rubric as Paper A, accepting relevant equipment-lending measures.

(b) Requirements [6] — use the same 3 + 3 rubric as Paper A; accept distinct lending functions and measurable quality criteria.

(c) Use-case diagram [6] — 2 for actors, 2 for use cases, 1 for boundary/associations, 1 for justified `<<include>>`/`<<extend>>` or their correct omission.

(d) Class diagram [12]

- 4: relevant classes (e.g. Member, CatalogueItem, Copy, Loan, Reservation and a justified service/controller).
- 4: valid relationships/multiplicities, including each Loan's one Member and one Copy; recognize that a Copy can have loan history but at most one active loan.
- 2: appropriate typed attributes and operations.
- 2: both one-active-loan-per-copy and two-active-loans-per-member constraints (1 each); consistent with the sequence/state views.

(e) Sequence diagram [8]

- 1: suitable lifelines (including the Librarian/checkout interface, lending service/domain/repository, and external Email Service as appropriate).
- 2: logically ordered librarian approval/checkout request and validations.
- 2: active-member/loan-limit checks and copy-status/hold-owner checks and safe allocation/claim of an Available copy or the member's held copy in the same transaction as loan persistence; respect any waiting reservation and retain the hold for an ineligible first waiter while refusing checkout. Active loans include OnLoan and Overdue.
- 1: Loan creation and persistence occur only when permitted; failed checkout creates no Loan and leaves any pre-existing hold unchanged. Successful checkout fulfils that member's held Reservation but must not create a queue Reservation just to claim a copy.
- 1: alternative ineligible/no-copy path.
- 1: confirmation is sent through the external email service only after commit.

(f) Loan state machine [4] — 1 for an initial node and meaningful states including a coherent returned outcome; 1 for successful issue into OnLoan and return from either OnLoan or Overdue; 1 for the due-date transition to Overdue; 1 for the renewal guard (one renewal only, from OnLoan not Overdue, and no unfulfilled reservation including a held copy). Accept OnLoan, Overdue and Returned with a renewalCount == 0 guard on an OnLoan self-transition that sets renewalCount = 1 and extends due time atomically with the reservation check, or a distinct Renewed state. Do not force a redundant state merely to increase the count.

Question 2 — Object modelling and object-oriented concepts (20 marks)

(a) Encapsulation, abstraction and class/object [4] — 1.5 for each OO principle (accurate definition plus suitable example), and 1 for the class/object distinction.

(b) Generalisation, inheritance and polymorphism [6]

- 2: distinguishes UML generalisation from an implementation inheritance mechanism.
- 1: valid, justified class hierarchy with an explicit policy assumption that does not conflict with the case.
- 2: shows the same operation/interface producing suitable type-specific behaviour.
- 1: explains a benefit or limitation (e.g. substitutability/reuse, or why inheritance should not be forced).

(c) `<<include>>` and `<<extend>>` [4]

- 2: correct required/reused versus conditional/optional behaviour distinction and relationship meaning/direction.
- 2: one valid example of each grounded in the lending system. Accept a well-justified alternative where the scenario genuinely supports it.

(d) Association, aggregation and composition [6] — 2 marks per relationship for correct meaning and a justified example/lifecycle assumption.

Question 3 — Dynamic and behavioural modelling (20 marks)

(a) State, event and operation [4] — 1 mark each for correct meaning; 1 mark for coherent examples/distinction.

(b) Copy state machine [8]

- 1: initial/terminal semantics.
- 2: suitable Copy states (e.g. Available, OnLoan, Maintenance, Retired, and Reserved for a waiting member); a returned or repaired copy must honour any waiting queue before becoming generally Available; cancellation advances to the next waiter or Available. No automatic expiry is specified; retirement requires no current loan or hold.
- 2: valid event-triggered transitions for issue, return, damage, repair and retirement.
- 1: an appropriate issue guard (e.g. active member, member below the loan limit, and the Copy is Available or held for that member).
- 1: distinguishes the Copy lifecycle from the Loan lifecycle; Overdue belongs to Loan, not Copy.
- 1: internally consistent, readable model.

(c) Diagram comparisons [4] — use the same 1 + 1 + 1 + 1 rubric as Paper A.

(d) Concurrency [4]

- 1: recognises competing requests for one physical Copy.
- 1: uses an appropriate guard/availability check.
- 1: uses an atomic checkout/lock/transaction or equivalent safe commit.
- 1: states the invariant: a Copy has no more than one active Loan at any time.

Question 4 — Unified Software Development Process (20 marks)

Use the same rubric as Paper A Question 4. Accept a phased pilot at one campus, an inventory-data quality risk, stakeholder testing and a staged transition/rollout as scenario-specific applications.

Question 5 — From analysis to design and architecture (20 marks)

(a) Analysis, design and traceability [4] — use the same 2 + 2 rubric as Paper A, grounded in member borrowing an available copy.

(b) Responsibility and design quality [8]

- 1: identifies low cohesion in `ResourceCentreSystem`.
- 1: identifies high coupling/direct UI-to-database access.
- 3: proposes sensible separation (e.g. catalogue/search service, LoanService, Loan/Copy/Member domain classes, repository, notification adapter).
- 1: links the redesign to maintainability, testing, security or changeability.
- 2: discusses one relevant strength and one limitation/risk of the object-oriented approach in this case (1 each).

(c) Architecture, components and CASE [8] — use the same 2 + 2 + 1 + 1 + 2 rubric as Paper A, grounded in catalogue, lending, member, persistence and notification responsibilities.

Final moderation checklist

Before approval, verify that each paper has Q1 = 40 marks, three selected Section B questions = 60 marks, and a total of 100 marks; the chosen duration is approved; scenario facts are internally consistent; diagrams are legible in print; and the marking guide is stored separately from candidate copies.